L28 — Double Ended Queue (Deque)

Unit: 2 | **Chapter:** Queues | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Define a deque and its operations at both ends
- Implement a deque using a circular array and a doubly linked list
- Apply deques to solve the sliding window maximum problem
- Compare deque with stack and queue

---

1. What is a Deque?

A **Double-Ended Queue (Deque)** is a generalization of both a stack and a queue that allows insertions and deletions at **both the front and the rear**.

```
push_front / pop_front
        ↕
     [  front ... rear  ]
                       ↕
              push_rear / pop_rear
```

---

2. Deque Operations

Operation	Description	Time
`push_front(item)`	Insert at front	O(1)
`push_rear(item)`	Insert at rear	O(1)
`pop_front()`	Remove from front	O(1)
`pop_rear()`	Remove from rear	O(1)
`peek_front()`	View front without removing	O(1)
`peek_rear()`	View rear without removing	O(1)
`is_empty()`	Check empty	O(1)
`size()`	Return count	O(1)

---

3. Deque Types

Type	Restriction
Input-Restricted Deque	Insertion allowed only at one end; deletion at both
Output-Restricted Deque	Deletion allowed only at one end; insertion at both
General Deque	Insert and delete at both ends

---

4. Circular Array Implementation

```
class Deque:
    def __init__(self, capacity=100):
        self.data = [None] * capacity
        self.capacity = capacity
        self.front = 0
        self._size = 0

    def _rear(self):
        return (self.front + self._size - 1) % self.capacity

    def is_empty(self):
        return self._size == 0

    def is_full(self):
        return self._size == self.capacity

    def push_rear(self, item):
        if self.is_full():
            raise OverflowError("Deque is full")
        rear = (self.front + self._size) % self.capacity
        self.data[rear] = item
        self._size += 1

    def push_front(self, item):
        if self.is_full():
            raise OverflowError("Deque is full")
        self.front = (self.front - 1) % self.capacity
        self.data[self.front] = item
        self._size += 1

    def pop_front(self):
        if self.is_empty():
            raise IndexError("Deque is empty")
        item = self.data[self.front]
        self.front = (self.front + 1) % self.capacity
        self._size -= 1
        return item

    def pop_rear(self):
        if self.is_empty():
            raise IndexError("Deque is empty")
        rear = self._rear()
        item = self.data[rear]
        self._size -= 1
        return item

    def peek_front(self):
        if self.is_empty():
            raise IndexError("Deque is empty")
        return self.data[self.front]
```

```
    def peek_rear(self):
        if self.is_empty():
            raise IndexError("Deque is empty")
        return self.data[self._rear()]
```

---

5. Doubly Linked List Implementation

```
class DLLNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DLLDeque:
    def __init__(self):
        self.head = None    # Front
        self.tail = None    # Rear
        self._size = 0

    def is_empty(self):
        return self.head is None

    def push_front(self, item):
        node = DLLNode(item)
        if self.is_empty():
            self.head = self.tail = node
        else:
            node.next = self.head
            self.head.prev = node
            self.head = node
        self._size += 1

    def push_rear(self, item):
        node = DLLNode(item)
        if self.is_empty():
            self.head = self.tail = node
        else:
            node.prev = self.tail
            self.tail.next = node
            self.tail = node
        self._size += 1

    def pop_front(self):
        if self.is_empty():
            raise IndexError("Deque is empty")
        item = self.head.data
        self.head = self.head.next
        if self.head:
            self.head.prev = None
        else:
```

```
            self.tail = None
        self._size -= 1
        return item

    def pop_rear(self):
        if self.is_empty():
            raise IndexError("Deque is empty")
        item = self.tail.data
        self.tail = self.tail.prev
        if self.tail:
            self.tail.next = None
        else:
            self.head = None
        self._size -= 1
        return item
```

---

6. Python's Built-in Deque

```
from collections import deque

d = deque()
d.append(10)         # push_rear
d.append(20)
d.appendleft(5)      # push_front → deque: [5, 10, 20]

print(d.popleft())   # pop_front → 5
print(d.pop())       # pop_rear → 20
print(d[0])          # peek_front → 10

# All deque operations are O(1)
# Implemented as doubly-linked list of fixed-size arrays (internal bloc
```

---

7. Application: Sliding Window Maximum (LeetCode #239)

Problem: Given array and window size k, find max in every window of k elements.

Brute force: O(nk) — for each window, scan all k elements.
Deque approach: O(n) — maintain a deque of indices in decreasing order of values.

```
from collections import deque

def max_sliding_window(nums, k):
    """
    Returns list of maximums for each sliding window of size k.
    Deque stores indices; deque front always holds index of current max
    """
    result = []
    dq = deque()   # Stores indices
```

```python
    for i in range(len(nums)):
        # Remove indices that are out of window
        while dq and dq[0] < i - k + 1:
            dq.popleft()

        # Remove smaller elements from rear (they'll never be max)
        while dq and nums[dq[-1]] < nums[i]:
            dq.pop()

        dq.append(i)

        # Window is fully formed
        if i >= k - 1:
            result.append(nums[dq[0]])

    return result

# Test
nums = [1, 3, -1, -3, 5, 3, 6, 7]
k = 3
print(max_sliding_window(nums, k))
# Output: [3, 3, 5, 5, 6, 7]
```

Trace for `[1, 3, -1, -3, 5, 3, 6, 7]`, k = 3:

i	nums[i]	Deque (indices)	Window	Max
0	1	[0]	—	—
1	3	[1] (remove 0: 1 > 3? No, remove 0: nums[0] = 1 < 3)	—	—
2	-1	[1, 2]	[1,3,-1]	3
3	-3	[1, 2, 3]	[3,-1,-3]	3
4	5	[4]	[-1,-3,5]	5
5	3	[4, 5]	[-3,5,3]	5
6	6	[6]	[5,3,6]	6
7	7	[7]	[3,6,7]	7

---

8. Deque vs Stack vs Queue

Feature	Stack	Queue	Deque
Insert front	✗	✗	✓
Insert rear	✓	✓	✓
Remove front	✓	✓	✓
Remove rear	✓	✗	✓
Access	LIFO	FIFO	Both

Summary

- Deque allows O(1) insertions and deletions at **both** front and rear.
- Two implementations: circular array (fixed size) and doubly linked list (dynamic).
- Python's `collections.deque` is highly optimized with O(1) amortized operations.
- Key application: **Sliding Window Maximum** — O(n) using a monotonic deque.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- LeetCode #239 — Sliding Window Maximum